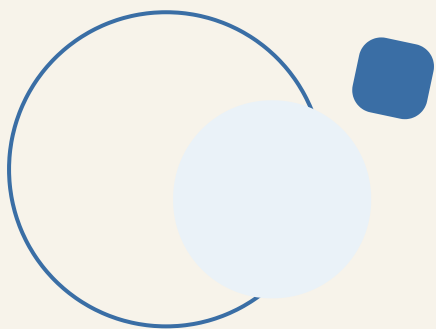




AI 开发 黑话翻译手册

第二册 | AI 开发与部署

程序员是怎么说的，普通人应该怎么理解。

把黑话翻译成成人话，再翻译成 AI 能执行的开发需求。

AI 开发黑话翻译手册

第二册 | AI 开发与部署

书名

AI 开发黑话翻译手册 (AI Development Jargon Translator)

副标题

程序员是怎么说的，普通人应该怎么理解

本册

第二册 · AI 开发与部署 · 共 94 个术语

版本

v1.0 · 2026 年

许可

本手册以 CC BY 4.0 协议开源，可自由分享、打印、用于学习与教学，注明出处即可。

本手册面向完全不懂技术的读者。所有解释都尽量用生活语言，不用黑话解释黑话。它同时是 “AI Product Clarifier” 技能的知识底座：当用户表示 “我不太理解” 时，可引用本手册中的解释。

怎么读这本书

这不是词典，是翻译器。每一页把一个技术黑话，翻译成你听得懂的话，再翻译成 AI 能执行的开发需求。

你不需要从头读到尾。遇到听不懂的词，翻到那一页，看一眼「一句话理解」，通常就够了。想更深入，再往下看例子和用法。

每一页长这样

①	分类标签	页面右上角，标明这个词属于产品 / 开发 / 部署 / AI 等哪一类。
②	中文术语	页面最大的字，就是这一页要讲的黑话。
③	英文术语	黑话的英文原文，方便你在真实软件里对照。
④	一句话理解	整页最重要的一句。看不懂别的，看这句就够。
⑤	生活中的例子	用你熟悉的日常事物，帮你秒懂。
⑥	程序员想表达什么	这个词在开发里到底指什么。
⑦	AI 会翻译成什么	你说人话，AI 实际会去做的开发动作。
⑧	什么时候需要 / 不需要	帮你判断这东西你到底要不要。
⑨	没有它会怎样	不做的后果，帮你权衡。
⑩	怎么告诉 AI	可以直接复制给 AI 的一句话。
⑪	记忆口诀	一句话记住它。

翻译逻辑：程序员语言 → 普通人语言 → AI 能执行的开发需求语言。

目录

01	前端 Frontend	001	24	Serverless Serverless	024
02	后端 Backend	002	25	部署 Deploy	025
03	全栈 Full-stack	003	26	上线 Go Live	026
04	客户端 Client	004	27	生产环境 Production	027
05	服务端 Server-side	005	28	测试环境 Staging	028
06	数据库 Database	006	29	开发环境 Development	029
07	本地存储 Local Storage	007	30	域名 Domain	030
08	云端存储 Cloud Storage	008	31	DNS DNS	031
09	SQLite SQLite	009	32	CDN CDN	032
10	Supabase Supabase	010	33	Cloudflare Cloudflare	033
11	PostgreSQL PostgreSQL	011	34	Vercel Vercel	034
12	MySQL MySQL	012	35	Netlify Netlify	035
13	Redis Redis	013	36	Docker Docker	036
14	Schema Schema	014	37	容器 Container	037
15	数据表 Table	015	38	镜像 Image	038
16	字段 Field / Column	016	39	HTTPS HTTPS	039
17	记录 Record / Row	017	40	SSL SSL/TLS	040
18	查询 Query	018	41	证书 Certificate	041
19	缓存 Cache	019	42	端口 Port	042
20	文件上传 File Upload	020	43	公网 Public Network	043
21	对象存储 Object Storage	021	44	内网 Private Network	044
22	服务器 Server	022	45	IP IP Address	045
23	云服务器 Cloud Server	023	46	防火墙 Firewall	046
			47	代理 Proxy	047

目录 (续)

48	Nginx Nginx	048	72	定时任务 Scheduled Task	072
49	API API	049	73	Cron Cron	073
50	接口 Interface	050	74	队列 Queue	074
51	REST REST API	051	75	消息队列 Message Queue	075
52	GraphQL GraphQL	052	76	限流 Rate Limiting	076
53	Webhook Webhook	053	77	负载均衡 Load Balancing	077
54	JSON JSON	054	78	CI Continuous Integration	078
55	SDK SDK	055	79	CD Continuous Delivery	079
56	环境变量 Environment Variable	056	80	构建 Build	080
57	日志 Log	057	81	打包 Bundle / Package	081
58	监控 Monitoring	058	82	编译 Compile	082
59	错误追踪 Error Tracking	059	83	依赖 Dependency	083
60	备份 Backup	060	84	包管理器 Package Manager	084
61	恢复 Restore	061	85	Node.js Node.js	085
62	认证 Authentication	062	86	Python Python	086
63	授权 Authorization	063	87	Next.js Next.js	087
64	Token Token	064	88	React React	088
65	Cookie Cookie	065	89	Vue Vue	089
66	Session Session	066	90	CLI Command-Line Interface	090
67	OAuth OAuth	067	91	终端 Terminal	091
68	JWT JWT	068	92	命令行 Command Line	092
69	邮件服务 Email Service	069	93	README README	093
70	短信服务 SMS Service	070	94	License License	094
71	支付接口 Payment Gateway	071			



一句话理解

用户能看见、能点的那一面。

- **生活中的例子**

餐厅里客人坐的用餐区，看得见摸得着。

- **程序员真正想表达什么**

浏览器里负责界面和交互的部分。

- **AI 会翻译成什么开发需求**

用 HTML/CSS 和 React 等做出用户可见的界面。

✓ **什么时候需要**

任何有界面的产品。

✗ **什么时候不需要**

纯后台脚本、无界面时。

- **没有它会怎样**

用户没有能看能操作的界面。

小白应该怎么告诉 AI

帮我做前端界面，展示__、让用户能点__。

“前端，是用户看见的那半边。”



一句话理解

藏在背后、干真正逻辑活儿的那一面。

- 生活中的例子

餐厅后厨，客人看不见，饭菜都在这做。

- 程序员真正想表达什么

负责数据处理、业务逻辑的服务器程序。

- AI 会翻译成什么开发需求

写服务器代码处理数据、规则和存储。

✓ 什么时候需要

涉及数据保存、账号、计算逻辑时。

✗ 什么时候不需要

纯静态展示、无逻辑时。

- 没有它会怎样

界面是空壳，点了没有任何真实结果。

小白应该怎么告诉 AI

帮我做后端，负责处理__并把数据存起来。

“后端，是幕后干活的那半边。”

全栈

Full-stack

开发



一句话理解

前端后端一个人全包。

- 生活中的例子

一个师傅既管点菜又管掌勺。

- 程序员真正想表达什么

同时负责界面和服务器逻辑的能力。

- AI 会翻译成什么开发需求

从界面到数据一整套一起做。

✓ 什么时候需要

小项目、一人或 AI 全包时。

✗ 什么时候不需要

大团队分工明确时。

- 没有它会怎样

两头对接不上，做出半拉子。

小白应该怎么告诉 AI

帮我从前端到后端做一整套能用的__。

“一个人，通吃两头。”

客户端

Client

开发



一句话理解

装在你设备上、由你这边运行的那一端。

- 生活中的例子

你手机里那个 App 就是客户端。

- 程序员真正想表达什么

运行在用户设备上、发起请求的一方。

- AI 会翻译成什么开发需求

开发在用户设备上运行的应用界面。

✓ 什么时候需要

要做手机/桌面应用时。

✗ 什么时候不需要

纯网页、无独立应用时。

- 没有它会怎样

用户没有属于自己设备上的入口。

小白应该怎么告诉 AI

帮我做一个客户端 App，用户装在手机上用。

“客户端，是你手里那一端。”

服务端

Server-side



一句话理解

远在机房、替所有人干活的那一端。

- 生活中的例子

银行的总系统，所有网点都连它。

- 程序员真正想表达什么

集中处理请求和数据的一方。

- AI 会翻译成什么开发需求

把核心逻辑放到服务器统一处理。

✓ 什么时候需要

多人共享数据、需集中处理时。

✗ 什么时候不需要

纯本地、不联网时。

- 没有它会怎样

每台设备各算各的，数据对不上。

小白应该怎么告诉 AI

把__的逻辑放到服务端统一处理。

﹂ 服务端，是替大家干活的那一端。 ﹂

数据库

Database



一句话理解

你希望下次打开时，之前的内容还在。

- 生活中的例子

微信聊天记录不会因为关掉微信就消失。

- 程序员真正想表达什么

有组织地长期保存数据的系统。

- AI 会翻译成什么开发需求

需要数据持久化，可能用 SQLite、Supabase 或数据库。

✓ 什么时候需要

任何要“记住”数据的功能。

✗ 什么时候不需要

数据用完即弃、无需保存时。

- 没有它会怎样

一关软件，用户的数据全没了。

小白应该怎么告诉 AI

我需要把__数据存起来，关掉再打开还在。

“数据库，是不会忘事的仓库。”

本地存储

Local Storage



一句话理解

把数据存在用户自己设备上。

- 生活中的例子

手机便签存在手机里，不联网也在。

- 程序员真正想表达什么

数据保存在客户端本地的方式。

- AI 会翻译成什么开发需求

把数据存到浏览器或设备本地。

✓ 什么时候需要

数据只属于本机、无需同步时。

✗ 什么时候不需要

多设备要同步数据时。

- 没有它会怎样

换台设备数据就没了。

小白应该怎么告诉 AI

把__存在用户本地就行，不用上传服务器。

“存在自己手里，快但带不走。”

云端存储

Cloud Storage



一句话理解

把数据存在网上，换设备也能拿到。

- 生活中的例子

照片存网盘，换手机登录还在。

- 程序员真正想表达什么

数据保存在远程服务器的方式。

- AI 会翻译成什么开发需求

把数据存到云端数据库或对象存储。

✓ 什么时候需要

要多设备同步、防丢失时。

✗ 什么时候不需要

纯本地小工具时。

- 没有它会怎样

换设备或丢设备，数据就找不回。

小白应该怎么告诉 AI

把__存到云端，让用户换设备也能看到。

∩ 存在云上，走到哪拿到哪。∩

SQLite

SQLite



一句话理解

一个装进单个文件里的小型数据库。

- 生活中的例子

像一本随身笔记本，轻便、随手就能用。

- 程序员真正想表达什么

无需单独服务、以文件形式运行的数据库。

- AI 会翻译成什么开发需求

用 SQLite 做轻量本地数据存储。

✓ 什么时候需要

小项目、单机、数据量不大时。

✗ 什么时候不需要

多人高并发、海量数据时。

- 没有它会怎样

小项目被迫上重型数据库，白折腾。

小白应该怎么告诉 AI

用 SQLite 存__就行，够用又简单。

“一个文件，就是一个小数据库。”

Supabase

Supabase



一句话理解

开箱即用的云数据库加后端套件。

- 生活中的例子

像即插即用的“后端全家桶”，省去自己搭。

- 程序员真正想表达什么

提供数据库、认证、存储的一体化后端服务。

- AI 会翻译成什么开发需求

用 Supabase 快速搞定数据库、登录和存储。

✓ 什么时候需要

想快速有个能用的后端时。

✗ 什么时候不需要

需要高度定制的复杂后端时。

- 没有它会怎样

自己从零搭后端，慢且容易出错。

小白应该怎么告诉 AI

用 Supabase 帮我搞定数据库和用户登录。

“后端全家桶，拿来就用。”

PostgreSQL

PostgreSQL



一句话理解

功能强大、正经可靠的大型数据库。

- 生活中的例子

像一座结构严谨的大仓库，能装能管。

- 程序员真正想表达什么

成熟稳定、功能丰富的关系型数据库。

- AI 会翻译成什么开发需求

用 PostgreSQL 做正式、可扩展的数据存储。

✓ 什么时候需要

数据重要、要长期可靠时。

✗ 什么时候不需要

只是小玩具项目时。

- 没有它会怎样

选了太简陋的存储，长大后扛不住。

小白应该怎么告诉 AI

用 PostgreSQL 存我们的核心数据。

👉 **正经数据，交给正经数据库。** 👈

MySQL

MySQL



一句话理解

最流行的老牌关系型数据库之一。

- **生活中的例子**

像用了很多年、大家都会用的老仓库。

- **程序员真正想表达什么**

广泛使用、生态成熟的关系型数据库。

- **AI 会翻译成什么开发需求**

用 MySQL 做通用数据存储。

✓ **什么时候需要**

常规网站、生态成熟需求时。

✗ **什么时候不需要**

极简单机小工具时。

- **没有它会怎样**

遇到问题时找不到成熟方案（其实很少）。

小白应该怎么告诉 AI

用 MySQL 来存我们的数据。

“老牌好用，用的人多。”

Redis

Redis



一句话理解

极快的临时数据存放处。

- 生活中的例子

像桌上的便签，随手记、拿得飞快，但不长久。

- 程序员真正想表达什么

内存级、高速读写的键值存储。

- AI 会翻译成什么开发需求

用 Redis 做缓存或临时高速数据。

✓ 什么时候需要

要极快读写、临时数据时。

✗ 什么时候不需要

要长期可靠保存时。

- 没有它会怎样

热点数据每次都慢慢查，系统卡。

小白应该怎么告诉 AI

用 Redis 缓存__，让访问更快。

“快是它的命，久不是它的活。”

Schema

Schema



一句话理解

数据长什么样、有哪些字段的设计图。

- **生活中的例子**

填表前先定好表格有哪几栏。

- **程序员真正想表达什么**

数据结构的定义和约束。

- **AI 会翻译成什么开发需求**

先定义好数据表结构再存数据。

- ✓ **什么时候需要**

设计数据怎么存时。

- ✗ **什么时候不需要**

数据结构极简、临时用时。

- **没有它会怎样**

数据乱七八糟，存进去取不出。

小白应该怎么告诉 AI

帮我设计一下__数据的结构（有哪些字段）。

“先画好格子，再往里填。”

数据表

Table



一句话理解

数据库里的一张表格，一类东西存一张。

- 生活中的例子

Excel 里“客户表”“订单表”各是一张。

- 程序员真正想表达什么

存储同类数据的结构化表格。

- AI 会翻译成什么开发需求

为每类数据建一张表并定义列。

- ✓ 什么时候需要

有多类数据要分开存时。

- ✗ 什么时候不需要

只有极少数据时。

- 没有它会怎样

各类数据混在一起，乱成一团。

小白应该怎么告诉 AI

帮我建一张__表，用来存所有的__。

‘一类东西，存一张表。’

字段

Field / Column



一句话理解

表格里的一栏，代表一种信息。

- 生活中的例子

通讯录里“姓名”“电话”各是一栏。

- 程序员真正想表达什么

数据表中的一列属性。

- AI 会翻译成什么开发需求

为每条数据定义要记录的属性列。

- ✓ 什么时候需要

设计一条数据要记什么时。

- ✗ 什么时候不需要

结构极简时。

- 没有它会怎样

该记的信息漏了，取不到。

小白应该怎么告诉 AI

每条__要记录这些字段：__、__、__。

“字段，是表里的一栏。”

记录

Record / Row



一句话理解

表格里的一行，代表一条具体数据。

- 生活中的例子

通讯录里“张三 138...”就是一条记录。

- 程序员真正想表达什么

数据表中的一行数据。

- AI 会翻译成什么开发需求

每新增一条数据就多一行记录。

- ✓ 什么时候需要

谈单条具体数据时。

- ✗ 什么时候不需要

谈结构而非数据时。

- 没有它会怎样

分不清一条条具体数据。

小白应该怎么告诉 AI

每次用户__，就新增一条记录。

“记录，是表里的一行。”

查询

Query



一句话理解

向数据库问“我要的数据在哪”。

- 生活中的例子

去仓库说“把所有红色的鞋拿给我”。

- 程序员真正想表达什么

从数据库按条件取出数据的操作。

- AI 会翻译成什么开发需求

编写查询语句按条件取数据。

- ✓ 什么时候需要

要从数据里找出符合条件的内容时。

- ✗ 什么时候不需要

无需读取数据时。

- 没有它会怎样

有数据却取不出想要的部分。

小白应该怎么告诉 AI

帮我查出所有满足__条件的__。

“查询，是跟仓库要货。”

缓存

Cache



一句话理解

把常用结果先存一份，下次直接拿，省得重算。

- 生活中的例子

常用的电话记在便签上，不用每次翻通讯录。

- 程序员真正想表达什么

临时保存结果以加速后续访问。

- AI 会翻译成什么开发需求

对高频数据加缓存减少重复计算。

- ✓ 什么时候需要

同样的数据被反复读取时。

- ✗ 什么时候不需要

数据每次都变、缓存没意义时。

- 没有它会怎样

每次都重算重查，系统又慢又累。

小白应该怎么告诉 AI

帮我给__加缓存，重复访问时更快。

常用的先存着，下次直接拿。

文件上传

File Upload



一句话理解

让用户把图片、文档等文件传进系统。

- **生活中的例子**

发邮件时点“添加附件”。

- **程序员真正想表达什么**

接收并保存用户提交文件的功能。

- **AI 会翻译成什么开发需求**

实现文件接收、校验与存储。

✓ **什么时候需要**

用户需要提交图片或文档时。

✗ **什么时候不需要**

纯文字、无文件时。

- **没有它会怎样**

用户没法把文件交给系统。

小白应该怎么告诉 AI

帮我加文件上传，让用户能传__文件。

“让文件也能交进来。”

对象存储

Object Storage



一句话理解

专门存图片、视频等大文件的云仓库。

- 生活中的例子

像一个超大网盘，专放大件行李。

- 程序员真正想表达什么

面向海量非结构化文件的存储服务。

- AI 会翻译成什么开发需求

用对象存储（如 S3）保存大文件。

✓ 什么时候需要

有大量图片视频要存时。

✗ 什么时候不需要

只存少量文字数据时。

- 没有它会怎样

大文件塞进数据库，又慢又贵。

小白应该怎么告诉 AI

把用户上传的图片视频存到对象存储里。

“大件行李，专库专放。”

服务器

Server



一句话理解

一台一直开着、替大家干活的电脑。

- 生活中的例子

银行大楼里那台从不关机的总机。

- 程序员真正想表达什么

长期运行、响应请求的计算机。

- AI 会翻译成什么开发需求

准备一台服务器来运行后端程序。

✓ 什么时候需要

程序要 24 小时在线服务时。

✗ 什么时候不需要

只在自己电脑本地跑时。

- 没有它会怎样

你一关电脑，别人就用不了。

小白应该怎么告诉 AI

我需要一台服务器来一直运行__。

≡ 服务器，是永不打烊的电脑。≡

云服务器

Cloud Server



一句话理解

租来的、放在网上的服务器。

- 生活中的例子

不用买房，直接租一间随时能用的办公室。

- 程序员真正想表达什么

云厂商按需提供的远程服务器。

- AI 会翻译成什么开发需求

在云平台开一台服务器部署程序。

- ✓ 什么时候需要

不想自己买机器、要弹性时。

- ✗ 什么时候不需要

有自建机房时（少见）。

- 没有它会怎样

自己买机器维护，又贵又麻烦。

小白应该怎么告诉 AI

帮我在云上开一台服务器来跑__。

“服务器也能租，用多少给多少。”

Serverless

Serverless



一句话理解

不用管服务器，代码来了就跑、跑完就走。

- 生活中的例子

像打车，用的时候才叫，不用养一辆车。

- 程序员真正想表达什么

无需管理服务器、按调用运行的模式。

- AI 会翻译成什么开发需求

把功能拆成函数，按需触发运行。

- ✓ 什么时候需要

流量不稳定、想省心省钱时。

- ✗ 什么时候不需要

需长时间常驻运行时。

- 没有它会怎样

为偶尔的流量白养一台机器。

小白应该怎么告诉 AI

用 Serverless 来跑__，按需触发就行。

‘用时才起，用完就散。’

部署

Deploy



一句话理解

把代码搬到线上，让别人能真正访问。

- 生活中的例子

把做好的菜从后厨端到客人桌上。

- 程序员真正想表达什么

把程序发布到运行环境对外提供服务。

- AI 会翻译成什么开发需求

把代码部署到服务器或平台上线。

- ✓ 什么时候需要

东西做好、要给用户用时。

- ✗ 什么时候不需要

还在本地开发调试时。

- 没有它会怎样

东西只在你电脑里，外人访问不了。

小白应该怎么告诉 AI

帮我把__部署上线，让用户能访问。

‘部署，是把菜端上桌。’

上线

Go Live

部署



一句话理解

正式对外开放，谁都能访问。

- **生活中的例子**

餐厅正式开门营业那一刻。

- **程序员真正想表达什么**

系统正式对外可用的状态。

- **AI 会翻译成什么开发需求**

完成部署并对外开放访问。

✓ **什么时候需要**

测试通过、准备迎接真实用户时。

✗ **什么时候不需要**

还没测稳定时。

- **没有它会怎样**

东西做好了却始终没真正开门。

小白应该怎么告诉 AI

测好之后就正式上线，让大家都能用。

‘上线，就是正式开门。’

生产环境

Production



一句话理解

真实用户正在用的那套正式系统。

- 生活中的例子

正在营业、真有客人的那家店。

- 程序员真正想表达什么

面向真实用户的正式运行环境。

- AI 会翻译成什么开发需求

把稳定版本发布到生产环境。

✓ 什么时候需要

面向真实用户提供服务时。

✗ 什么时候不需要

还在开发测试阶段时。

- 没有它会怎样

拿真实用户当小白鼠，一出错影响所有人。

小白应该怎么告诉 AI

把稳定版发到生产环境给真实用户用。

≡ 生产环境，动真格的地方。≡

测试环境

Staging



一句话理解

上线前用来彩排的“仿真店”。

- 生活中的例子

开业前的试营业，只请自己人来走一遍。

- 程序员真正想表达什么

接近生产、用于验证的预发布环境。

- AI 会翻译成什么开发需求

先在测试环境验证，再发到生产。

- ✓ 什么时候需要

上线前想安全验证时。

- ✗ 什么时候不需要

极小项目、直接上线时。

- 没有它会怎样

没彩排就开门，真实用户面前翻车。

小白应该怎么告诉 AI

先发测试环境验证，没问题再上线。

“先彩排，再开门。”

开发环境

Development



一句话理解

你自己电脑上边写边试的地盘。

- **生活中的例子**

厨师在自家厨房练手，不影响餐厅。

- **程序员真正想表达什么**

开发者本地编写和调试的环境。

- **AI 会翻译成什么开发需求**

在本地开发环境编码和自测。

✓ **什么时候需要**

写代码、随便试错时。

✗ **什么时候不需要**

要给别人看真实效果时。

- **没有它会怎样**

在真实系统上乱改，容易搞砸。

小白应该怎么告诉 AI

先在开发环境把__做出来自测。

‘自家厨房，随便练手。’

域名

Domain

部署



一句话理解

网站的门牌地址，好记的那串网址。

- 生活中的例子

记 taobao.com，而不是一串数字地址。

- 程序员真正想表达什么

映射到服务器的可读网址。

- AI 会翻译成什么开发需求

买一个域名并解析到服务器。

- ✓ 什么时候需要

要让用户用网址访问时。

- ✗ 什么时候不需要

只在本地测试时。

- 没有它会怎样

用户只能记一串难记的数字地址。

小白应该怎么告诉 AI

帮我配一个域名 __，指向我的网站。

💡 域名，是网站好记的门牌。💡

DNS

DNS



一句话理解

把好记的网址翻译成机器认的地址。

- 生活中的例子

像通讯录，把“妈妈”翻译成她的手机号。

- 程序员真正想表达什么

域名到 IP 地址的解析系统。

- AI 会翻译成什么开发需求

配置 DNS 记录把域名指向服务器。

✓ 什么时候需要

配置域名、让访问生效时。

✗ 什么时候不需要

不用域名时。

- 没有它会怎样

域名指不到服务器，网址打不开。

小白应该怎么告诉 AI

帮我配好 DNS，让 __ 这个域名能打开我的站。

“DNS，是网络的通讯录。”



一句话理解

把内容放到离用户最近的地方，加载更快。

- **生活中的例子**

全国连锁便利店，就近就能买，不用跑总部。

- **程序员真正想表达什么**

分布式缓存加速内容分发的网络。

- **AI 会翻译成什么开发需求**

用 CDN 加速静态资源的访问。

✓ **什么时候需要**

用户遍布各地、想加载更快时。

✗ **什么时候不需要**

只有本地少量用户时。

- **没有它会怎样**

远方用户打开又慢又卡。

小白应该怎么告诉 AI

帮我用 CDN 加速图片和页面加载。

≡ 内容摆到用户家门口，才快。≡

Cloudflare

Cloudflare



一句话理解

一个免费好用的加速加防护平台。

- **生活中的例子**

给网站请了个又能加速又能挡坏人的保安。

- **程序员真正想表达什么**

提供 CDN、DNS、防护的一体化平台。

- **AI 会翻译成什么开发需求**

用 Cloudflare 做加速、DNS 和防护。

✓ **什么时候需要**

想省心地加速和防护网站时。

✗ **什么时候不需要**

极简本地项目时。

- **没有它会怎样**

网站慢、还容易被攻击。

小白应该怎么告诉 AI

用 Cloudflare 帮我加速并保护网站。

‘又快又安全，还免费。’



一句话理解

前端项目一键上线的平台。

- **生活中的例子**

像一键发朋友圈，写完点一下就发布了。

- **程序员真正想表达什么**

面向前端的自动化部署托管平台。

- **AI 会翻译成什么开发需求**

把前端项目连上 Vercel 自动部署。

✓ **什么时候需要**

做 Next.js 等前端、想快速上线时。

✗ **什么时候不需要**

复杂后端为主时。

- **没有它会怎样**

上线要手动折腾一堆配置。

小白应该怎么告诉 AI

用 Vercel 帮我把前端一键部署上线。

ˋ 写完一点，立刻上线。 ˊ

Netlify

Netlify



一句话理解

另一个网站一键部署平台。

- 生活中的例子

和 Vercel 类似的“一键发布”按钮。

- 程序员真正想表达什么

静态站点与前端的托管部署平台。

- AI 会翻译成什么开发需求

把静态站点连上 Netlify 自动部署。

✓ 什么时候需要

部署静态网站、想省事时。

✗ 什么时候不需要

重后端项目时。

- 没有它会怎样

静态站上线也得手动搞半天。

小白应该怎么告诉 AI

用 Netlify 帮我把这个网站部署上线。

‘静态网站，一键就上。’

Docker

Docker



一句话理解

把程序和它需要的一切打包成一个标准箱子。

- 生活中的例子

把整套厨房装进集装箱，搬到哪都能开火。

- 程序员真正想表达什么

用容器封装程序及其运行环境。

- AI 会翻译成什么开发需求

用 Docker 打包程序，保证到哪都能一样运行。

- ✓ 什么时候需要

想让程序在任何机器都跑得一样时。

- ✗ 什么时候不需要

极简单机小脚本时。

- 没有它会怎样

换台机器就“在我这儿明明能跑”。

小白应该怎么告诉 AI

用 Docker 把__打包，保证到哪都能跑。

👉 打包成箱，到哪都一样。👈

容器

Container

部署



一句话理解

一个装着程序和环境的独立小盒子。

- 生活中的例子

一个自带水电的集装箱房，独立又标准。

- 程序员真正想表达什么

隔离运行程序及依赖的轻量单元。

- AI 会翻译成什么开发需求

把服务放进容器独立运行。

✓ 什么时候需要

要隔离、可复制地运行服务时。

✗ 什么时候不需要

极简本地运行时。

- 没有它会怎样

多个程序互相打架、环境冲突。

小白应该怎么告诉 AI

把__放进一个容器里独立运行。

﹂ 各装各的箱，互不打扰。 ﹂

镜像

Image



一句话理解

容器的“出厂模板”，照它就能造出一样的箱子。

- 生活中的例子

像做菜的标准配方，照着做味道都一样。

- 程序员真正想表达什么

构建容器的只读模板。

- AI 会翻译成什么开发需求

构建镜像用于创建一致的容器。

- ✓ 什么时候需要

要批量、可复制地部署时。

- ✗ 什么时候不需要

只跑一次的临时脚本时。

- 没有它会怎样

每台机器装出来的环境都不一样。

小白应该怎么告诉 AI

帮我打一个镜像，照它就能部署出一样的服务。

∩ 镜像，是箱子的出厂模板。∩

HTTPS

HTTPS



一句话理解

网址前带小锁，数据传输被加密。

- **生活中的例子**

寄挂号信，中途没人能拆看。

- **程序员真正想表达什么**

加密传输的安全 HTTP 协议。

- **AI 会翻译成什么开发需求**

为网站配置 HTTPS 加密访问。

✓ **什么时候需要**

任何涉及用户数据的网站。

✗ **什么时候不需要**

纯本地测试时。

- **没有它会怎样**

数据明文传输，容易被偷看。

小白应该怎么告诉 AI

帮我给网站配上 HTTPS，让访问是加密的。

“带锁的网址，才安全。”

SSL

SSL/TLS



一句话理解

让网址能上锁的那套加密技术。

- **生活中的例子**

给信封加密封条的那套工艺。

- **程序员真正想表达什么**

实现传输加密的安全协议。

- **AI 会翻译成什么开发需求**

配置 SSL/TLS 启用加密通信。

- ✓ **什么时候需要**

要开启 HTTPS 时。

- ✗ **什么时候不需要**

无需加密时。

- **没有它会怎样**

网站没法加密，浏览器还会警告。

小白应该怎么告诉 AI

帮我配好 SSL，让网站能用 HTTPS。

“SSL，是给网络上锁的手艺。”

证书

Certificate



一句话理解

证明“这网站是真的、可信的”的凭证。

- 生活中的例子

像营业执照，证明这家店是正规的。

- 程序员真正想表达什么

验证网站身份、启用加密的数字凭证。

- AI 会翻译成什么开发需求

申请并安装 SSL 证书。

- ✓ 什么时候需要

要开启 HTTPS、取得信任时。

- ✗ 什么时候不需要

纯本地时。

- 没有它会怎样

浏览器提示“不安全”，用户不敢用。

小白应该怎么告诉 AI

帮我申请并配好网站证书，去掉不安全提示。

“有证在手，用户才信。”

端口

Port



一句话理解

一台机器上不同服务各走的一道门。

- 生活中的例子

一栋楼有很多门，各部门走各的门。

- 程序员真正想表达什么

标识主机上不同服务的编号通道。

- AI 会翻译成什么开发需求

为不同服务分配和开放端口。

- ✓ 什么时候需要

一台机器跑多个服务时。

- ✗ 什么时候不需要

只跑单一服务时。

- 没有它会怎样

服务抢同一道门，互相冲突。

小白应该怎么告诉 AI

把__服务放在 __ 端口上运行。

“一个服务，走一道门。”

公网

Public Network

部署



一句话理解

整个互联网都能访问到的地址。

- **生活中的例子**

临街的店面，谁路过都能进。

- **程序员真正想表达什么**

可被外部互联网访问的网络。

- **AI 会翻译成什么开发需求**

把服务开放到公网供外部访问。

✓ **什么时候需要**

要让外部用户访问时。

✗ **什么时候不需要**

只给内部用时。

- **没有它会怎样**

外部用户根本连不上你的服务。

小白应该怎么告诉 AI

把__放到公网，让外面的人能访问。

“公网，是临街的门面。”

内网

Private Network



一句话理解

只有自己能访问的内部网络。

- 生活中的例子

公司内部才能进的办公区。

- 程序员真正想表达什么

仅限内部访问的隔离网络。

- AI 会翻译成什么开发需求

把敏感服务限制在内网。

✓ 什么时候需要

服务只给内部、要安全时。

✗ 什么时候不需要

要对外开放时。

- 没有它会怎样

本该保密的服务暴露在外。

小白应该怎么告诉 AI

把__放在内网，只让内部访问。

“内网，是自家院子。”

IP

IP Address

1 2
3 4

一句话理解

每台联网设备在网络里的门牌号。

- 生活中的例子

像每个家庭都有的门牌地址。

- 程序员真正想表达什么

标识网络设备的地址编号。

- AI 会翻译成什么开发需求

为服务器配置和使用 IP 地址。

- ✓ 什么时候需要

配置网络、定位设备时。

- ✗ 什么时候不需要

无需直接接触底层网络时。

- 没有它会怎样

找不到设备在哪，连不上。

小白应该怎么告诉 AI

我的服务器 IP 是__，帮我把域名指过去。

“IP，是设备的门牌号。”

防火墙

Firewall



一句话理解

挡在门口、决定谁能进的安全门卫。

- 生活中的例子

小区门口的保安，可疑的人不让进。

- 程序员真正想表达什么

按规则拦截网络访问的安全机制。

- AI 会翻译成什么开发需求

配置防火墙只放行必要的访问。

✓ 什么时候需要

服务对外、要防攻击时。

✗ 什么时候不需要

纯本地隔离环境时。

- 没有它会怎样

谁都能进，容易被攻击。

小白应该怎么告诉 AI

帮我配防火墙，只开放__端口，其余全挡。

ˋ 把好门，坏人进不来。ˊ

代理

Proxy

部署



一句话理解

替你转发请求的中间人。

- 生活中的例子

跑腿代购，你不出面，他替你去买。

- 程序员真正想表达什么

转发并中转网络请求的中间层。

- AI 会翻译成什么开发需求

用代理转发或聚合请求。

- ✓ 什么时候需要

要隐藏来源、统一转发时。

- ✗ 什么时候不需要

直连即可时。

- 没有它会怎样

请求无法统一转发或隐藏。

小白应该怎么告诉 AI

帮我配个代理，把请求统一转发到__。

代理，是替你跑腿的中间人。

Nginx

Nginx



一句话理解

站在最前面分发请求的门口调度员。

- 生活中的例子

商场大门的引导员，把人流指向各楼层。

- 程序员真正想表达什么

高性能的反向代理与静态服务器。

- AI 会翻译成什么开发需求

用 Nginx 做请求转发和静态托管。

- ✓ 什么时候需要

要转发请求、托管静态资源时。

- ✗ 什么时候不需要

极简单机应用时。

- 没有它会怎样

请求进来没人调度，服务混乱。

小白应该怎么告诉 AI

用 Nginx 把请求转发到__，并托管静态文件。

≡ 门口有人调度，秩序才好。≡

API

API



一句话理解

两个软件之间的电话线，让它们互相传话。

- **生活中的例子**

餐厅点菜：你（顾客）通过服务员和后厨沟通，服务员就是 API。

- **程序员真正想表达什么**

系统对外提供的、可被调用的接口。

- **AI 会翻译成什么开发需求**

设计接口让前端或别的系统调用你的功能。

- ✓ **什么时候需要**

不同系统或前后端要交换数据时。

- ✗ **什么时候不需要**

完全独立、不与外界交互时。

- **没有它会怎样**

各软件之间无法沟通、传不了数据。

小白应该怎么告诉 AI

帮我做一个 API，让__能拿到__数据。

“API，是软件之间的电话线。”

接口

Interface



一句话理解

一个对外约定好的“怎么找我办事”的入口。

- 生活中的例子

银行柜台窗口，照规矩递材料就能办事。

- 程序员真正想表达什么

对外暴露、约定输入输出的调用点。

- AI 会翻译成什么开发需求

定义清晰的输入输出让别人调用。

- ✓ 什么时候需要

要让别人用你的功能时。

- ✗ 什么时候不需要

纯内部、不对外时。

- 没有它会怎样

别人不知道怎么用你的功能。

小白应该怎么告诉 AI

帮我定义一个接口，输入__返回__。

“接口，是对外办事的窗口。”

REST

REST API



一句话理解

一种最常见的接口设计风格。

- 生活中的例子

像标准化的快递单，大家都按同一格式填。

- 程序员真正想表达什么

基于 HTTP 资源的接口约定风格。

- AI 会翻译成什么开发需求

按 REST 规范设计增删改查接口。

✓ 什么时候需要

做常规增删改查接口时。

✗ 什么时候不需要

需求特殊、REST 不合适时。

- 没有它会怎样

接口五花八门，别人难对接。

小白应该怎么告诉 AI

帮我用 REST 风格做__的增删改查接口。

👉 照标准格式来，谁都好对接。👈

GraphQL

GraphQL



一句话理解

让调用方自己挑要哪些数据的接口方式。

- 生活中的例子

自助餐，想吃什么自己夹，不多不少。

- 程序员真正想表达什么

客户端按需查询数据的接口规范。

- AI 会翻译成什么开发需求

用 GraphQL 让前端按需取数据。

✓ 什么时候需要

前端要灵活取不同字段时。

✗ 什么时候不需要

接口简单、固定返回时。

- 没有它会怎样

要么取太多、要么反复加接口。

小白应该怎么告诉 AI

用 GraphQL，让前端自己挑要哪些字段。

“要什么自己点，不多给。”

Webhook

Webhook



一句话理解

有事发生时，系统主动打电话通知你。

- 生活中的例子

快递到了，快递员主动打电话，不用你一直问。

- 程序员真正想表达什么

事件触发时向指定地址推送通知。

- AI 会翻译成什么开发需求

配置 Webhook 在事件发生时回调你的接口。

✓ 什么时候需要

要在别的系统发生事时被通知时。

✗ 什么时候不需要

自己定时查就够时。

- 没有它会怎样

只能不停轮询，既慢又费。

小白应该怎么告诉 AI

当__发生时，用 Webhook 通知我的系统。

≡ 有事它喊你，不用你一直问。≡

JSON

JSON



一句话理解

一种软件之间传数据的通用格式。

- **生活中的例子**

像大家都看得懂的标准填表格式。

- **程序员真正想表达什么**

轻量的结构化数据交换格式。

- **AI 会翻译成什么开发需求**

用 JSON 作为接口的数据格式。

- ✓ **什么时候需要**

系统间传结构化数据时。

- ✗ **什么时候不需要**

传纯文件或二进制时。

- **没有它会怎样**

数据格式各不相同，对不上。

小白应该怎么告诉 AI

接口数据都用 JSON 格式返回。

“JSON，是软件间的通用表格。”

SDK

开发

SDK



一句话理解

别人做好的工具包，帮你少写很多代码。

- 生活中的例子

宜家家具的配件包，照着装就行。

- 程序员真正想表达什么

封装好的开发工具与函数集合。

- AI 会翻译成什么开发需求

引入 SDK 直接调用现成能力。

✓ 什么时候需要

要接入第三方服务时。

✗ 什么时候不需要

功能简单、无需依赖时。

- 没有它会怎样

从零造轮子，又慢又易错。

小白应该怎么告诉 AI

用__的官方 SDK 帮我快速接入它的功能。

👉 现成工具包，拿来就用。👈

环境变量

Environment Variable



一句话理解

把密钥、配置等放在代码外面的开关。

- 生活中的例子

保险箱密码不写在门上，另外记在安全处。

- 程序员真正想表达什么

运行时注入的外部配置项。

- AI 会翻译成什么开发需求

把密钥和配置放进环境变量而非硬编码。

✓ 什么时候需要

有密钥、有不同环境配置时。

✗ 什么时候不需要

无任何敏感配置时。

- 没有它会怎样

密钥写死在代码里，一泄露就完蛋。

小白应该怎么告诉 AI

把__的密钥放到环境变量里，别写死在代码里。

≡ 秘密别写墙上，另存别处。≡

日志

Log



一句话理解

程序运行时留下的“流水账”。

- 生活中的例子

监控录像，出事了能回看当时发生了什么。

- 程序员真正想表达什么

记录运行过程和事件的输出。

- AI 会翻译成什么开发需求

打印并保存运行日志便于排查。

✓ 什么时候需要

要排查问题、追溯过程时。

✗ 什么时候不需要

极简一次性脚本时。

- 没有它会怎样

出问题两眼一抹黑，不知哪错了。

小白应该怎么告诉 AI

帮我给__加日志，出问题能回看发生了什么。

‘留好流水账，出事好查。’

监控

Monitoring

部署



一句话理解

时刻盯着系统好不好、有没有异常。

- 生活中的例子

ICU 的心电监护，一有异常就报警。

- 程序员真正想表达什么

持续观测系统健康与指标的机制。

- AI 会翻译成什么开发需求

接入监控，异常时及时告警。

✓ 什么时候需要

线上服务要保稳定时。

✗ 什么时候不需要

本地一次性运行时。

- 没有它会怎样

系统挂了都不知道，用户先发现。

小白应该怎么告诉 AI

帮我加监控，__出问题能及时报警。

‘盯着点，才能早发现。’

错误追踪

Error Tracking



一句话理解

自动记录程序哪里报错、怎么错的。

- 生活中的例子

汽车故障灯，亮了就知道哪出问题。

- 程序员真正想表达什么

捕获并汇总运行时错误的工具。

- AI 会翻译成什么开发需求

接入错误追踪自动收集报错。

✓ 什么时候需要

线上要快速定位 bug 时。

✗ 什么时候不需要

极小项目时。

- 没有它会怎样

用户报错你却不知道错在哪。

小白应该怎么告诉 AI

帮我接入错误追踪，自动收集线上报错。

‘错在哪，让它自己招。’

备份

Backup



一句话理解

给重要数据多存一份，以防万一。

- 生活中的例子

重要文件多存一份到网盘，丢了还有。

- 程序员真正想表达什么

定期复制数据以防丢失。

- AI 会翻译成什么开发需求

设置定期自动备份数据。

✓ 什么时候需要

数据重要、丢不起时。

✗ 什么时候不需要

数据可随时重建时。

- 没有它会怎样

一旦出事，数据全丢无法挽回。

小白应该怎么告诉 AI

帮我给__设置定期自动备份。

“多存一份，睡得安稳。”

恢复

Restore



一句话理解

出事后，用备份把数据还原回来。

- 生活中的例子

手机坏了，用云备份把资料装回新机。

- 程序员真正想表达什么

从备份还原数据到可用状态。

- AI 会翻译成什么开发需求

实现从备份恢复数据的流程。

- ✓ 什么时候需要

出故障要救回数据时。

- ✗ 什么时候不需要

无重要数据时。

- 没有它会怎样

有备份却不会用，等于白备。

小白应该怎么告诉 AI

帮我做好从备份恢复__数据的流程。

“备了要能还，才算真备。”

认证

Authentication



一句话理解

确认“你是不是你本人”。

- 生活中的例子

进门先刷脸，确认是你才放行。

- 程序员真正想表达什么

验证用户身份真实性的过程。

- AI 会翻译成什么开发需求

实现登录校验确认用户身份。

✓ 什么时候需要

有账号、要确认身份时。

✗ 什么时候不需要

人人可用、无账号时。

- 没有它会怎样

冒名顶替随便进，账号不安全。

小白应该怎么告诉 AI

帮我做认证，用户登录时确认他是本人。

“先证明是你，再让你进。”

授权

Authorization



一句话理解

确认“你能不能做这件事”。

- 生活中的例子

有门禁卡能进楼，但进不了机房。

- 程序员真正想表达什么

在身份确认后判定操作权限。

- AI 会翻译成什么开发需求

按角色控制用户能访问什么。

- ✓ 什么时候需要

不同用户能做的事不同时。

- ✗ 什么时候不需要

所有人权限相同时。

- 没有它会怎样

登录后什么都能干，越权出事。

小白应该怎么告诉 AI

登录后按角色控制谁能做__、谁不能。

≡ 认证问你是谁，授权问你能干嘛。≡

Token

Token



一句话理解

登录后发给你的一张“通行证”。

- 生活中的例子

进游乐园换的手环，凭它随便玩。

- 程序员真正想表达什么

代表身份和权限的访问凭证。

- AI 会翻译成什么开发需求

用 Token 验证后续请求的身份。

- ✓ 什么时候需要

接口要识别登录用户时。

- ✗ 什么时候不需要

无登录状态时。

- 没有它会怎样

每次操作都要重新登录，太烦。

小白应该怎么告诉 AI

登录后发一个 Token，之后凭它访问接口。

‘一证在手，畅通无阻。’

Cookie

Cookie



一句话理解

网站存在你浏览器里的小纸条。

- **生活中的例子**

商店在你会员卡上盖的章，下次认得你。

- **程序员真正想表达什么**

浏览器保存的小段状态数据。

- **AI 会翻译成什么开发需求**

用 Cookie 保存登录状态等信息。

- ✓ **什么时候需要**

要在浏览器记住用户状态时。

- ✗ **什么时候不需要**

纯无状态接口时。

- **没有它会怎样**

每次访问都像第一次，记不住你。

小白应该怎么告诉 AI

用 Cookie 记住用户的登录状态。

“网站塞给你的小纸条。”

Session

Session



一句话理解

你这一次登录期间的“会话”记录。

- 生活中的例子

进网吧开的一台机，下机就结束。

- 程序员真正想表达什么

服务端维护的用户登录会话状态。

- AI 会翻译成什么开发需求

用 Session 维持登录期间的状态。

- ✓ 什么时候需要

要维持一段登录期状态时。

- ✗ 什么时候不需要

纯 Token 无状态时。

- 没有它会怎样

系统记不住你还在登录中。

小白应该怎么告诉 AI

用 Session 维持用户这次登录的状态。

👉 登录到退出，就是一次会话。👈

OAuth

OAuth



一句话理解

用微信/谷歌账号一键登录别的应用。

- 生活中的例子

拿身份证去别处办事，不用重新办证。

- 程序员真正想表达什么

授权第三方登录而不暴露密码的协议。

- AI 会翻译成什么开发需求

接入 OAuth 支持第三方账号登录。

- ✓ 什么时候需要

想让用户用现成账号快速登录时。

- ✗ 什么时候不需要

只需自建账号时。

- 没有它会怎样

用户嫌注册麻烦，直接走了。

小白应该怎么告诉 AI

帮我接入用微信/谷歌一键登录（OAuth）。

👉 借个现成身份，一键登录。👈

JWT

JWT



一句话理解

一种自带信息、防篡改的通行证。

- 生活中的例子

带防伪水印的门票，验票不用打电话核对。

- 程序员真正想表达什么

可自校验的令牌格式。

- AI 会翻译成什么开发需求

用 JWT 做无状态的身份令牌。

- ✓ 什么时候需要

接口要无状态验证身份时。

- ✗ 什么时候不需要

简单会话即可时。

- 没有它会怎样

每次都要查库验身份，慢。

小白应该怎么告诉 AI

用 JWT 做登录令牌，接口能自己验证。

👉 一张带防伪的自证门票。👈

邮件服务

Email Service



一句话理解

让系统能自动发邮件。

- **生活中的例子**

注册后自动收到的那封验证邮件。

- **程序员真正想表达什么**

对接邮件发送能力的服务。

- **AI 会翻译成什么开发需求**

接入邮件服务发送验证或通知邮件。

- ✓ **什么时候需要**

要发验证码、通知邮件时。

- ✗ **什么时候不需要**

完全不发邮件时。

- **没有它会怎样**

无法给用户发验证或通知。

小白应该怎么告诉 AI

帮我接入邮件服务，注册时给用户发验证邮件。

“让系统会自己发邮件。”

短信服务

SMS Service



一句话理解

让系统能自动发短信验证码。

- 生活中的例子

登录时收到的那条“验证码 6 位数”。

- 程序员真正想表达什么

对接短信发送能力的服务。

- AI 会翻译成什么开发需求

接入短信服务发送验证码或通知。

- ✓ 什么时候需要

要手机验证码、短信提醒时。

- ✗ 什么时候不需要

不需要短信时。

- 没有它会怎样

没法用手机号验证用户。

小白应该怎么告诉 AI

帮我接入短信服务，发手机验证码。

“让系统会自己发短信。”

支付接口

Payment Gateway



一句话理解

帮你安全收钱的中间通道。

- 生活中的例子

商店里的刷卡机，安全地把钱收进来。

- 程序员真正想表达什么

对接第三方完成收款的接口。

- AI 会翻译成什么开发需求

接入支付宝/微信/Stripe 等收款。

✓ 什么时候需要

要向用户收钱时。

✗ 什么时候不需要

完全免费时。

- 没有它会怎样

想收钱却没有安全的收款渠道。

小白应该怎么告诉 AI

帮我接入支付接口，让用户能安全付款。

💡 收钱这事，交给专业通道。💡

定时任务

Scheduled Task



一句话理解

让系统到点自动干某件事。

- 生活中的例子

定好每天早 7 点自动响的闹钟。

- 程序员真正想表达什么

按时间自动触发的后台任务。

- AI 会翻译成什么开发需求

实现按计划自动执行的任务。

- ✓ 什么时候需要

要定期自动跑某事时。

- ✗ 什么时候不需要

只手动触发时。

- 没有它会怎样

该定时做的事得靠人记着手动做。

小白应该怎么告诉 AI

帮我设一个定时任务，每天__自动执行__。

、 到点自动办，不用惦记。、

Cron

Cron



一句话理解

专门用来安排定时任务的时间表写法。

- 生活中的例子

像列车时刻表，规定每趟几点发车。

- 程序员真正想表达什么

以特定表达式设定周期任务的机制。

- AI 会翻译成什么开发需求

用 Cron 表达式配置任务执行周期。

- ✓ 什么时候需要

要精确控制运行周期时。

- ✗ 什么时候不需要

只跑一次时。

- 没有它会怎样

定时规则说不清，任务乱跑。

小白应该怎么告诉 AI

用 Cron 设置每__执行一次__。

👉 Cron，是任务的时刻表。👈

队列

Queue



一句话理解

让任务排好队，一个个来，不挤爆。

- 生活中的例子

食堂排队打饭，一个个来，不哄抢。

- 程序员真正想表达什么

按顺序缓冲和处理任务的结构。

- AI 会翻译成什么开发需求

用队列把任务排队逐个处理。

- ✓ 什么时候需要

任务多、要削峰有序处理时。

- ✗ 什么时候不需要

任务少、即时处理即可时。

- 没有它会怎样

任务一拥而上，系统被压垮。

小白应该怎么告诉 AI

把__任务放进队列排队处理，别一次全上。

“排好队，一个个来。”

消息队列

Message Queue



一句话理解

系统之间靠“留言板”异步传递任务。

- 生活中的例子

把便条投进信箱，对方有空再取来办。

- 程序员真正想表达什么

系统间异步传递消息的中间件。

- AI 会翻译成什么开发需求

用消息队列解耦系统、异步处理。

✓ 什么时候需要

多系统要异步、解耦协作时。

✗ 什么时候不需要

单系统同步处理即可时。

- 没有它会怎样

系统紧耦合，一个卡全卡。

小白应该怎么告诉 AI

用消息队列让__和__异步传递任务。

“留个言，各忙各的。”

限流

Rate Limiting



一句话理解

限制访问速度，防止被打爆。

- **生活中的例子**

景区限流，每小时只放一定人数进。

- **程序员真正想表达什么**

控制单位时间请求量的保护机制。

- **AI 会翻译成什么开发需求**

对接口做限流防止滥用和过载。

- ✓ **什么时候需要**

怕被刷、被压垮时。

- ✗ **什么时候不需要**

内部低频使用时。

- **没有它会怎样**

被恶意刷爆，服务瘫痪。

小白应该怎么告诉 AI

帮我给__接口加限流，防止被刷爆。

ˋ 限个流,别被挤爆。 ˊ

负载均衡

Load Balancing



一句话理解

把访问分摊到多台机器，谁都不累垮。

- 生活中的例子

多个收银台分流顾客，队都不长。

- 程序员真正想表达什么

在多台服务器间分配流量的机制。

- AI 会翻译成什么开发需求

用负载均衡把请求分到多台服务器。

- ✓ 什么时候需要

流量大、单机扛不住时。

- ✗ 什么时候不需要

流量很小时。

- 没有它会怎样

全压在一台机器上，一挂全挂。

小白应该怎么告诉 AI

帮我加负载均衡，把流量分到多台服务器。

≡ 多开几个窗口，队就短了。≡

Continuous Integration



一句话理解

每次改代码就自动检查有没有出问题。

- 生活中的例子

每写完一页就自动帮你检查错别字。

- 程序员真正想表达什么

代码提交后自动构建与测试的流程。

- AI 会翻译成什么开发需求

配置 CI 在提交时自动跑测试。

- ✓ 什么时候需要

多人协作、要保证代码质量时。

- ✗ 什么时候不需要

个人小脚本时。

- 没有它会怎样

问题攒到最后才发现，难收拾。

小白应该怎么告诉 AI

帮我配 CI，每次提交代码自动跑测试检查。

“边写边查，问题早现形。”

Continuous Delivery



一句话理解

代码通过检查后自动发布上线。

- 生活中的例子

打印完自动装订、直接寄出。

- 程序员真正想表达什么

自动化把通过的代码交付到环境。

- AI 会翻译成什么开发需求

配置 CD 在测试通过后自动部署。

- ✓ 什么时候需要

想让上线又快又稳时。

- ✗ 什么时候不需要

极简单机项目时。

- 没有它会怎样

每次上线都得手动折腾易出错。

小白应该怎么告诉 AI

帮我配 CD，测试通过后自动部署上线。

“过了检查，自动送上线。”

构建

Build



一句话理解

把写的代码加工成能运行的成品。

- **生活中的例子**

把面粉鸡蛋做成能吃的蛋糕。

- **程序员真正想表达什么**

把源码编译打包成可运行产物。

- **AI 会翻译成什么开发需求**

运行构建生成可部署的产物。

- ✓ **什么时候需要**

代码要变成可运行成品时。

- ✗ **什么时候不需要**

解释型脚本直接运行时。

- **没有它会怎样**

代码是原料，没加工不能用。

小白应该怎么告诉 AI

帮我把项目构建成可以部署的成品。

把原料，做成能上桌的成品。

打包

Bundle / Package



一句话理解

把一堆文件压成一个方便分发的整体。

- 生活中的例子

搬家把零碎装进几个大箱子，好搬。

- 程序员真正想表达什么

把代码和资源合并压缩成分发包。

- AI 会翻译成什么开发需求

把项目打包成可发布的文件。

✓ 什么时候需要

要分发、部署项目时。

✗ 什么时候不需要

本地零散运行时。

- 没有它会怎样

文件散一地，难分发难加载。

小白应该怎么告诉 AI

帮我把项目打包成一个可发布的文件。

“零碎装箱，方便搬运。”

编译

Compile



一句话理解

把人写的代码翻译成机器能执行的语言。

- 生活中的例子

把中文稿翻译成外国人能读的版本。

- 程序员真正想表达什么

把源代码转换为机器码的过程。

- AI 会翻译成什么开发需求

编译源码为可执行程序。

- ✓ 什么时候需要

用编译型语言时。

- ✗ 什么时候不需要

用解释型脚本时。

- 没有它会怎样

机器读不懂源码，跑不起来。

小白应该怎么告诉 AI

帮我把这段代码编译成能运行的程序。

翻译成机器话，它才听得懂。

依赖

Dependency



一句话理解

你的程序要用到的别人写好的现成模块。

- 生活中的例子

做菜要用到别人生产的酱油、味精。

- 程序员真正想表达什么

项目运行所需的第三方库。

- AI 会翻译成什么开发需求

声明并安装项目所需依赖。

✓ 什么时候需要

要用现成库、不重复造轮子时。

✗ 什么时候不需要

完全自给自足时（少见）。

- 没有它会怎样

缺了它，程序根本跑不起来。

小白应该怎么告诉 AI

这个项目依赖__，帮我装好。

👉 站在别人肩上，少造轮子。👈

包管理器

Package Manager



一句话理解

帮你自动下载、管理依赖的工具。

- 生活中的例子

像跑腿助手，你列清单它帮你把料买齐。

- 程序员真正想表达什么

管理项目依赖安装与版本的工具。

- AI 会翻译成什么开发需求

用 npm/pip 等安装管理依赖。

- ✓ 什么时候需要

项目有多个依赖时。

- ✗ 什么时候不需要

无任何依赖时。

- 没有它会怎样

得手动一个个找库装，又乱又累。

小白应该怎么告诉 AI

用包管理器帮我装好项目需要的所有依赖。

“列个清单，它帮你买齐。”

Node.js

Node.js



一句话理解

让 JavaScript 也能在服务器上跑的环境。

- 生活中的例子

让原本只在教室用的工具，也能搬去工厂用。

- 程序员真正想表达什么

服务端运行 JavaScript 的运行时。

- AI 会翻译成什么开发需求

用 Node.js 写后端服务。

✓ 什么时候需要

想用 JS 写前后端时。

✗ 什么时候不需要

指定用别的语言时。

- 没有它会怎样

没法用 JS 写服务器端逻辑。

小白应该怎么告诉 AI

用 Node.js 帮我写后端。

“让前端语言也能管后厨。”

Python

Python



一句话理解

简单好学、AI 领域最常用的编程语言。

- **生活中的例子**

像一门语法简单、上手快的外语。

- **程序员真正想表达什么**

通用、易读、AI 生态丰富的语言。

- **AI 会翻译成什么开发需求**

用 Python 写脚本、后端或 AI 逻辑。

✓ **什么时候需要**

做数据、AI、脚本时。

✗ **什么时候不需要**

指定用别的语言时。

- **没有它会怎样**

缺少最顺手的 AI 开发语言。

小白应该怎么告诉 AI

用 Python 帮我实现__。

“简单好学，AI 都爱用。”

Next.js

Next.js



一句话理解

做现代网站的一套流行框架。

- 生活中的例子

像装修的整套工具包，省得零件到处找。

- 程序员真正想表达什么

基于 React 的全栈网页框架。

- AI 会翻译成什么开发需求

用 Next.js 快速搭建现代网站。

- ✓ 什么时候需要

做正经网站、要快又好时。

- ✗ 什么时候不需要

极简单页时。

- 没有它会怎样

从零搭网站，配置一堆很折腾。

小白应该怎么告诉 AI

用 Next.js 帮我搭这个网站。

“现代网站的整套工具包。”

React

React



一句话理解

把界面拆成一块块积木来搭的前端工具。

- **生活中的例子**

用乐高积木拼东西，块块可复用。

- **程序员真正想表达什么**

组件化构建用户界面的前端库。

- **AI 会翻译成什么开发需求**

用 React 把界面拆成组件来开发。

✓ **什么时候需要**

做交互丰富的界面时。

✗ **什么时候不需要**

极简静态页时。

- **没有它会怎样**

界面代码乱成一团，难维护。

小白应该怎么告诉 AI

用 React 帮我做这个交互界面。

👉 把界面拆成积木来搭。👈



一句话理解

另一套简单友好的前端框架。

- **生活中的例子**

和 React 类似的另一盒界面积木。

- **程序员真正想表达什么**

渐进式的前端界面框架。

- **AI 会翻译成什么开发需求**

用 Vue 构建前端界面。

✓ **什么时候需要**

想要上手更简单的前端时。

✗ **什么时候不需要**

团队已定用别的框架时。

- **没有它会怎样**

缺少组织界面的顺手工具。

小白应该怎么告诉 AI

用 Vue 帮我做这个前端界面。

“上手友好的界面积木。”

Command-Line Interface



一句话理解

靠打字命令来操作的黑框界面。

- **生活中的例子**

不用鼠标点，直接对着窗口下命令办事。

- **程序员真正想表达什么**

通过命令行操作程序的方式。

- **AI 会翻译成什么开发需求**

提供命令行方式操作工具。

✓ **什么时候需要**

做开发者工具、要自动化时。

✗ **什么时候不需要**

面向普通用户、要图形界面时。

- **没有它会怎样**

没有可脚本化的高效操作入口。

小白应该怎么告诉 AI

帮我做一个命令行工具，输入__就能__。

“打几行命令，活就办了。”

终端

Terminal

开发



一句话理解

那个可以敲命令的黑色窗口。

- 生活中的例子

和电脑直接对话的“对讲机”。

- 程序员真正想表达什么

输入命令、查看输出的文本窗口。

- AI 会翻译成什么开发需求

在终端里运行命令操作项目。

- ✓ 什么时候需要

跑命令、装依赖、部署时。

- ✗ 什么时候不需要

纯图形操作时。

- 没有它会怎样

很多开发操作没法高效完成。

小白应该怎么告诉 AI

在终端里帮我运行__这条命令。

“终端，是和电脑对话的窗口。”

命令行

Command Line

开发



一句话理解

在终端里敲的那一行行指令。

- 生活中的例子

对着助手说“帮我把灯关了”这类具体指令。

- 程序员真正想表达什么

通过文本指令让程序执行操作。

- AI 会翻译成什么开发需求

用命令行指令完成开发操作。

- ✓ 什么时候需要

要精确、可重复地操作时。

- ✗ 什么时候不需要

点点鼠标就够时。

- 没有它会怎样

复杂操作难以自动化重复。

小白应该怎么告诉 AI

帮我写出完成__需要敲的命令行指令。

“一行命令，胜过点十下。”

README

README



一句话理解

项目的“说明书”，告诉别人这是什么、怎么用。

- 生活中的例子

新家电盒子里那张“使用说明”。

- 程序员真正想表达什么

介绍项目用途与用法的说明文档。

- AI 会翻译成什么开发需求

写一份 README 说明项目和用法。

- ✓ 什么时候需要

项目要给别人看、开源时。

- ✗ 什么时候不需要

纯私人临时脚本时。

- 没有它会怎样

别人打开项目一脸懵，不会用。

小白应该怎么告诉 AI

帮我写一份 README，介绍这个项目和怎么用。

ˋ 先读我，再用我。 ˊ

License

License



一句话理解

告诉别人这东西能不能用、怎么用的许可说明。

- **生活中的例子**

书上的“版权所有/可否转载”声明。

- **程序员真正想表达什么**

规定代码使用与分发权利的许可。

- **AI 会翻译成什么开发需求**

为项目选择并添加开源许可证。

- ✓ 什么时候需要**

开源、要讲清使用权时。

- ✗ 什么时候不需要**

纯私有不公开时。

- **没有它会怎样**

别人不敢用，怕侵权。

小白应该怎么告诉 AI

帮我给项目加一个 __ 开源许可证。

💡 **讲清能不能用，别人才敢用。** 💡

术语索引

按中文（拼音 / 笔画大致排序）

API	API	049	README	README	093
CD	Continuous Delivery	079	REST	REST API	051
CDN	CDN	032	React	React	088
CI	Continuous Integration	078	Redis	Redis	013
CLI	Command-Line Interface	090	SDK	SDK	055
Cloudflare	Cloudflare	033	SQLite	SQLite	009
Cookie	Cookie	065	SSL	SSL/TLS	040
Cron	Cron	073	Schema	Schema	014
DNS	DNS	031	Serverless	Serverless	024
Docker	Docker	036	Session	Session	066
GraphQL	GraphQL	052	Supabase	Supabase	010
HTTPS	HTTPS	039	Token	Token	064
IP	IP Address	045	Vercel	Vercel	034
JSON	JSON	054	Vue	Vue	089
JWT	JWT	068	Webhook	Webhook	053
License	License	094	上线	Go Live	026
MySQL	MySQL	012	云服务器	Cloud Server	023
Netlify	Netlify	035	云端存储	Cloud Storage	008
Next.js	Next.js	087	代理	Proxy	047
Nginx	Nginx	048	依赖	Dependency	083
Node.js	Node.js	085	全栈	Full-stack	003
OAuth	OAuth	067	公网	Public Network	043
PostgreSQL	PostgreSQL	011	内网	Private Network	044
Python	Python	086			

按中文（拼音 / 笔画大致排序）（续）

前端 Frontend	001	构建 Build	080
包管理器 Package Manager	084	查询 Query	018
后端 Backend	002	测试环境 Staging	028
命令行 Command Line	092	消息队列 Message Queue	075
域名 Domain	030	环境变量 Environment Variable	056
备份 Backup	060	生产环境 Production	027
字段 Field / Column	016	监控 Monitoring	058
定时任务 Scheduled Task	072	短信服务 SMS Service	070
客户端 Client	004	端口 Port	042
容器 Container	037	终端 Terminal	091
对象存储 Object Storage	021	缓存 Cache	019
开发环境 Development	029	编译 Compile	082
恢复 Restore	061	认证 Authentication	062
打包 Bundle / Package	081	记录 Record / Row	017
授权 Authorization	063	证书 Certificate	041
接口 Interface	050	负载均衡 Load Balancing	077
支付接口 Payment Gateway	071	邮件服务 Email Service	069
数据库 Database	006	部署 Deploy	025
数据表 Table	015	错误追踪 Error Tracking	059
文件上传 File Upload	020	镜像 Image	038
日志 Log	057	队列 Queue	074
服务器 Server	022	防火墙 Firewall	046
服务端 Server-side	005	限流 Rate Limiting	076
本地存储 Local Storage	007		

按英文 (A-Z)

API API	049	Deploy 部署	025
Authentication 认证	062	Development 开发环境	029
Authorization 授权	063	DNS DNS	031
Backend 后端	002	Docker Docker	036
Backup 备份	060	Domain 域名	030
Build 构建	080	Email Service 邮件服务	069
Bundle / Package 打包	081	Environment Variable 环境变量	056
Cache 缓存	019	Error Tracking 错误追踪	059
CDN CDN	032	Field / Column 字段	016
Certificate 证书	041	File Upload 文件上传	020
Client 客户端	004	Firewall 防火墙	046
Cloud Server 云服务器	023	Frontend 前端	001
Cloud Storage 云端存储	008	Full-stack 全栈	003
Cloudflare Cloudflare	033	Go Live 上线	026
Command Line 命令行	092	GraphQL GraphQL	052
Command-Line Interface CLI	090	HTTPS HTTPS	039
Compile 编译	082	Image 镜像	038
Container 容器	037	Interface 接口	050
Continuous Delivery CD	079	IP Address IP	045
Continuous Integration CI	078	JSON JSON	054
Cookie Cookie	065	JWT JWT	068
Cron Cron	073	License License	094
Database 数据库	006	Load Balancing 负载均衡	077
Dependency 依赖	083		

按英文 (A-Z) (续)

Local Storage 本地存储	007	React React	088
Log 日志	057	README README	093
Message Queue 消息队列	075	Record / Row 记录	017
Monitoring 监控	058	Redis Redis	013
MySQL MySQL	012	REST API REST	051
Netlify Netlify	035	Restore 恢复	061
Next.js Next.js	087	Scheduled Task 定时任务	072
Nginx Nginx	048	Schema Schema	014
Node.js Node.js	085	SDK SDK	055
OAuth OAuth	067	Server 服务器	022
Object Storage 对象存储	021	Server-side 服务端	005
Package Manager 包管理器	084	Serverless Serverless	024
Payment Gateway 支付接口	071	Session Session	066
Port 端口	042	SMS Service 短信服务	070
PostgreSQL PostgreSQL	011	SQLite SQLite	009
Private Network 内网	044	SSL/TLS SSL	040
Production 生产环境	027	Staging 测试环境	028
Proxy 代理	047	Supabase Supabase	010
Public Network 公网	043	Table 数据表	015
Python Python	086	Terminal 终端	091
Query 查询	018	Token Token	064
Queue 队列	074	Vercel Vercel	034
Rate Limiting 限流	076	Vue Vue	089
		Webhook Webhook	053

写在最后

黑话不是为了拒人于门外，它只是行业里的省事说法。当你能把它翻译成人话，你就已经能和 AI、和程序员平等对话了。

这本手册的野心很小：让你在听到一个词时，不再点头假装懂，而是能说出“哦，原来它就是那个意思”。它的野心也很大：让不懂技术的人，也能用 AI 把想法做成真正的产品。

如果某个词的解释你觉得还能更清楚，欢迎在 GitHub 上提出。这本书会一直更新。

本册是《AI 开发黑话翻译手册》三册中的第二册。三册合起来，覆盖从产品想法、到开发部署、再到 AI 编程与 GitHub 的完整旅程。

翻译逻辑始终不变：**程序员语言 → 普通人语言 → AI 能执行的开发需求语言**。